

Process & Methodology

Student Study Edition · Golden Master design · 11 structured lessons with original course visuals, examples, activities, and study aids.

01 · Process and Methodology

02 · Development Challenges

03 · Process Models Overview

04 · Waterfall Model

05 · Prototyping Model

06 · Evolutionary Development

TOPIC 04 · STUDY MAP

Contents

Use this map to move through the topic in the same lesson order as The Educator course pages.

01	Process and Methodology	3
02	Development Challenges	6
03	Process Models Overview	9
04	Waterfall Model	11
05	Prototyping Model	14
06	Evolutionary Development	16
07	Spiral Model	18
08	Agile Process Models	21
09	Choosing the Right Model	24
10	Agile Methods	27
11	Software Paradigms	30

TOPIC 04 · LESSON 01 · 1 OF 11 LESSONS

TOPIC 04 · CHAPTER 2 · LESSON 01

Process and Methodology

A process gives the development journey its structure. A methodology tells the team how to perform the work inside that structure.

LEARNING FOCUS

Distinguish software process from methodology and explain how phases, artifacts, milestones, criteria, steps and techniques work together.

PRACTICE LAB · TOPIC 04**Mission 04 — Choose the Development Strategy**

Analyze uncertainty, coordination, change, stability and risk; distinguish process from methodology; compare lifecycle models; then choose and justify the development strategy that best manages the project context.

CORE LECTURE

Route versus recipe

 Process = what & when

A set of interrelated activities carried out in phases. Activities produce artifacts; phase completion establishes milestones; artifacts can become inputs to later work.

 Methodology = how

An implementation of a process or phase that specifies the detailed steps, representations, procedures, techniques and flows used to perform the work.

METHODOLOGY ANATOMY

What makes “how” precise?



Input

What enters the step?



Entry Criteria

When may work begin?



Steps & Techniques

How is work performed?



Artifact / Output

What is produced?



Exit Criteria

When is the step complete?

Software example: Requirements analysis may begin when stakeholder scope is available, apply interviews/modeling techniques, produce a requirements specification or models, and exit when agreed quality and review criteria are satisfied.

WORKED ANALOGY

Building permit

The process defines stages such as submit → review → approve. The methodology defines the forms, checklist, representations, review procedure, acceptance criteria and handoffs used inside each stage.

MEMORY HOOK

**Process = lifecycle structure. Methodology = execution detail.
Good engineering needs both.**

STUDY BOUNDARY Core lecture content ends above.

The section below is added clarification to make the concept easier to understand and revise.

EXAM LENS

Distinguish nearby concepts

If the question asks...

Which phases happen, what artifacts flow between them, or where milestones occur → think **process**.

If the question asks...

Which steps, procedures, techniques, representations or entry/exit conditions are used → think **methodology**.

TOPIC 04 · LESSON 02 · 2 OF 11 LESSONS

TOPIC 04 · CHAPTER 2 · LESSON 02

Development Challenges

Different process models exist because projects face different combinations of uncertainty, coordination, change, constraints and technology.

LEARNING FOCUS

Explain the project and system pressures that make a disciplined software process necessary and connect those pressures to process-model choices.

CORE LECTURE

Why coding alone is not enough

- ✓ Projects may last years while future customer needs remain uncertain.
- ✓ Multiple departments, teams and locations must divide, coordinate and integrate work.
- ✓ Teams may use different processes, methods and tools.
- ✓ Systems must satisfy many requirements and constraints that can change over time.
- ✓ Hardware, software, third-party components, languages, platforms and machines must interoperate.

PRESSURE → ENGINEERING RESPONSE

Each challenge creates a different need

**Uncertainty**

Need learning and feedback before committing too much.

**Many Teams**

Need coordination, interfaces,

**Change**

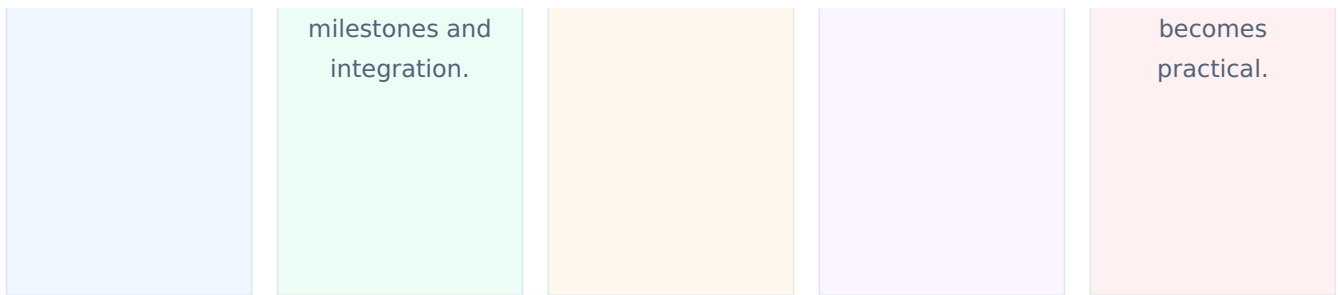
Need a process that can absorb evolving requirements.

**High Risk**

Need explicit analysis and reduction of uncertainty.

**Stability**

When requirements are stable, stronger up-front planning



Why do different process models exist?

Uncertainty
→ feedback

Teams →
coordination

Ch
adapt

Key bridge: process models are alternative engineering responses to different project pressures. The next lesson compares those responses.

WORKED EXAMPLE

Smart-city platform

Transit, payments, sensors, mobile apps, vendors and operations teams cannot be coordinated by coding alone. Their uncertainty, interfaces and changing constraints determine what kind of process discipline the project needs.

REMEMBER

Do not ask “Which model is best?” Ask “Which project pressure must the process manage?”

STUDY BOUNDARY Core lecture content ends above.

The section below is added clarification to make the concept easier to understand and revise.

EXAM LENS

Translate symptoms into process needs

Scenario clue

Unclear customer needs usually signal a need for early learning and feedback.

Scenario clue

Distributed teams, strict constraints or high consequences signal stronger coordination, evidence or risk control.

TOPIC 04 · LESSON 03 · 3 OF 11 LESSONS

TOPIC 04 · CHAPTER 2 · LESSON 03

Process Models Overview

The same project can be organized using different process logics. The model changes when feedback, commitment, and risk are handled.

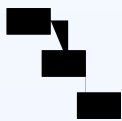
LEARNING FOCUS

Describe the different software development models: waterfall, prototype, evolutionary, spiral, and agile.

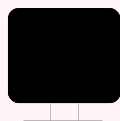
CORE LECTURE · FIVE PROCESS LOGICS

Five ways to organize development

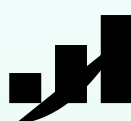
Each model answers the same question differently: **when should the team commit, learn, reduce risk, and deliver?** Use the visual hook on each card to remember its dominant logic.

**Waterfall**

Major development phases proceed in a planned sequence, with strong emphasis on completing one stage before the next.

SEQUENCE**Prototype**

Build a model early to expose unclear requirements, test ideas, and learn before committing to the full solution.

LEARN**Evolutionary**

Working versions grow progressively; each version extends or improves the system that will eventually become the production product.

GROW**Spiral**

Development proceeds through repeated loops driven by identifying, analyzing, and reducing the most important risks.

REDUCE RISK**Agile**

Work is organized into short repeated delivery and feedback cycles so the team can adapt as needs and priorities change.

ADAPT

Memory hook: Waterfall **flows** → Prototype **learns** → Evolutionary **grows** → Spiral **de-risks** → Agile **adapts**.

WORKED EXAMPLE

Campus mobile app

Stable regulatory reporting may suit plan-driven work; uncertain student workflows benefit from prototypes and iterative delivery.

REMEMBER

Choose a model by the problem's uncertainty, risk, coordination, and feedback needs—not by fashion.

STUDY BOUNDARY Core lecture content ends above.

The section below is added clarification to make the concept easier to understand and revise.

ADDED CLARIFICATION

Connect it to engineering practice

Ask yourself

What problem does this concept solve, and what would go wrong if the team ignored it?

Exam lens

Be able to define the concept, distinguish it from nearby concepts, and apply it to a short scenario.

TOPIC 04 · LESSON 04 · 4 OF 11 LESSONS

TOPIC 04 · CHAPTER 2 · LESSON 04

Waterfall Model

Waterfall organizes development into identified sequential phases with clear milestones and documentation.

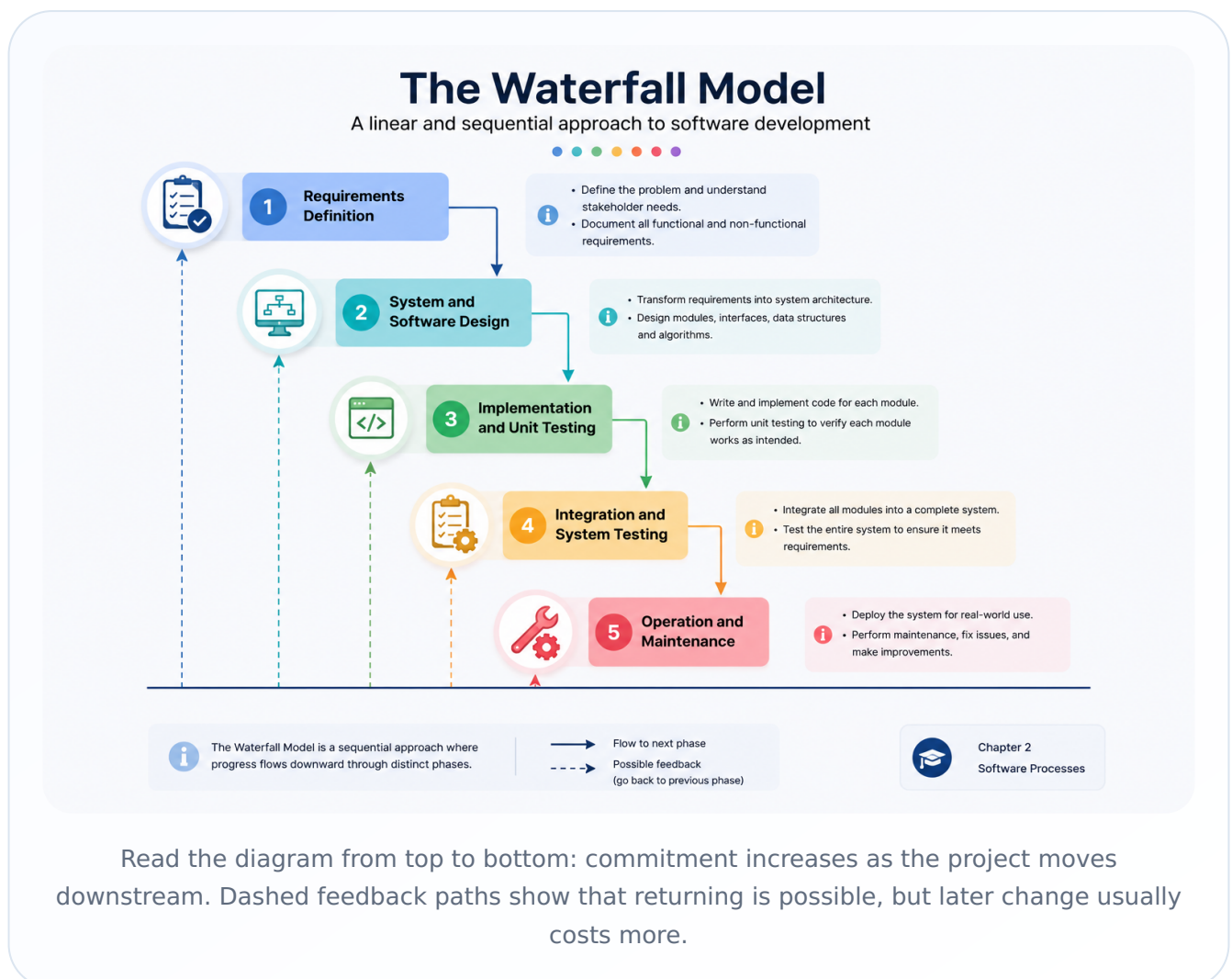
LEARNING FOCUS

Explain the Waterfall sequence, its phase-completion logic, its strengths and limitation, and when a plan-driven model is an appropriate choice.

CORE LECTURE · VISUAL ANCHOR

Waterfall is a staged commitment

Each major phase produces knowledge and deliverables that support the next phase. The uploaded diagram below is the primary visual for this model: it shows the downward progression, the purpose of each phase, and the possibility of feedback to earlier stages.



Sequence rule: a phase is expected to be substantially complete before the next phase proceeds. This gives Waterfall its clarity—and also explains why late change can be expensive.

WHY TEAMS CHOOSE IT—OR AVOID IT

Three hooks to remember



Strength · Order

Clear planning, milestones, documentation and coordination make progress visible and can help large or distributed engineering efforts stay synchronized.



Limitation · Late Change

Once downstream phases are underway, changing an earlier assumption can force expensive rework across design, implementation and testing.



Best Fit · Stability

Waterfall is most suitable when requirements are well understood, major changes are unlikely, and the organization values staged approvals and predictable documentation.

WORKED EXAMPLE

Government reporting system

If mandatory forms, regulations and interfaces are stable, a plan-driven sequence can simplify approvals, documentation and coordination. The same model becomes less attractive if policy changes every few weeks.

REMEMBER

Waterfall trades adaptability for staged commitment, visibility and coordination.

STUDY BOUNDARY Core lecture content ends above.

The section below is added clarification to make the concept easier to understand and revise.

ADDED CLARIFICATION

Connect it to engineering practice

Decision cue

Ask: “How stable are the requirements, and how costly would late change be?” Stable requirements strengthen the case for Waterfall.

Exam lens

Be able to list the major phases, explain why the model is sequential, identify its main strength and drawback, and justify whether it fits a short scenario.

TOPIC 04 · LESSON 05 · 5 OF 11 LESSONS

TOPIC 04 · CHAPTER 2 · LESSON 05

Prototyping Model

A prototype is built early to learn what users actually need before committing to the complete solution.

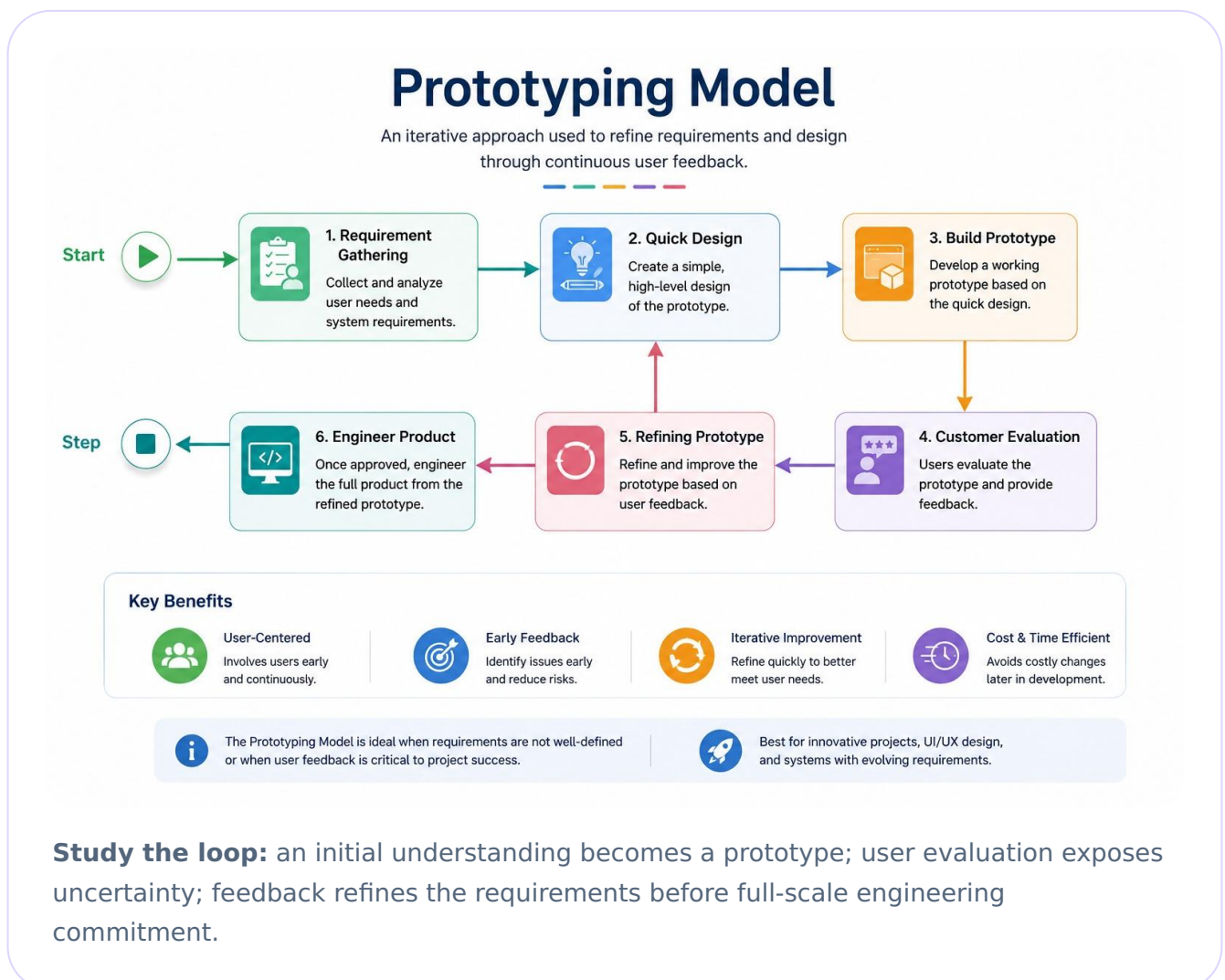
LEARNING FOCUS

Explain how prototyping reduces requirements uncertainty, distinguish throwaway from evolutionary prototyping, and recognize why a prototype is primarily a learning instrument.

CORE LECTURE · VISUAL ANCHOR

Prototype to learn before you commit

When requirements are uncertain, build an early model, let users react to it, and use feedback to refine what the system should become.



TWO PROTOTYPE PATHS

What happens after the learning?

Throwaway / Rapid Prototype

Build quickly to explore requirements or usability, capture the learning, then discard it and engineer the production solution properly.

Evolutionary Prototype

Start with a working version and deliberately improve it until it grows into the production system.

WORKED EXAMPLE

Clinic appointment interface

A clickable prototype reveals that users need family-member booking, clearer slot filters, and a simpler cancellation path before backend implementation begins.

REMEMBER

Prototype to learn. A prototype may look complete long before the engineering behind it is production-ready.

STUDY BOUNDARY Core lecture content ends above.

The section below is added clarification to make the concept easier to understand and revise.

ADDED CLARIFICATION

Connect it to engineering practice

Decision cue

Use prototyping when the biggest risk is **not knowing what users really need**.

Exam lens

Explain the feedback loop and distinguish throwaway from evolutionary prototyping.

TOPIC 04 · LESSON 06 · 6 OF 11 LESSONS

TOPIC 04 · CHAPTER 2 · LESSON 06

Evolutionary Development

Build something useful, learn from real use, and let the production system grow through successive versions.

Learning focus

Explain how working versions evolve toward production, distinguish evolution from throwaway prototyping, and recognize the engineering debt uncontrolled growth can create.

CORE LECTURE · FOUR ANCHORS

Think of the product as something that grows

**Start Small**

Implement a useful subset of capability.

**Use It**

Users evaluate real working versions.

**Learn**

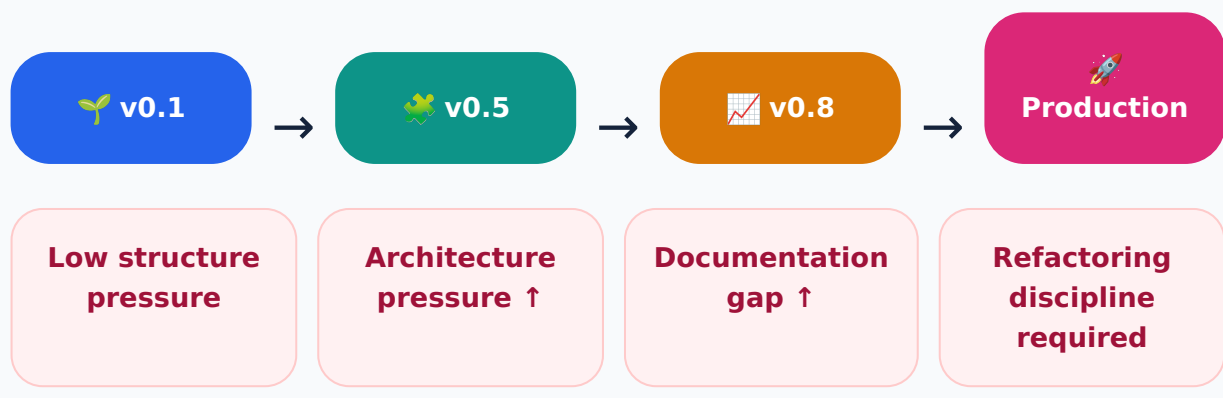
Feedback shapes later versions.

**Grow**

The same product evolves toward production.

VISUAL GROWTH MODEL

Capability grows — but unmanaged debt can grow too



The trade-off: rapid evolution improves learning and flexibility, but ad-hoc growth can make the process hard to trace, weaken structure, increase tool dependency and make documentation expensive.

CRITICAL DISTINCTION

Prototype or evolving product?



Throwaway Prototype

Build → learn → discard. Keep the knowledge, not the prototype.



Evolutionary Development

Build → learn → improve → production. Keep and strengthen the working system.

WORKED EXAMPLE

Research analytics tool

Scientists begin with dataset upload and one analysis, then add visualizations, collaboration and automation across working releases. As capability grows, the team must deliberately refactor architecture and maintain documentation so the evolving prototype does not become a fragile production system.

MEMORY HOOK · →

Evolution means growth, not replacement. Flexibility is valuable only when engineering discipline grows with the product.

STUDY BOUNDARY Core lecture content ends above.
The section below is added clarification.

EXAM LENS

Recognize the pattern

Evolutionary

Users repeatedly receive working versions and the same product grows over time.

Throwaway

The early model is intentionally discarded after requirements are learned.

TOPIC 04 · LESSON 07 · 7 OF 11 LESSONS

TOPIC 04 · CHAPTER 2 · LESSON 07

Spiral Model

Iteration is visible in the shape. **Risk-driven commitment** is the idea that defines the model.

LEARNING FOCUS

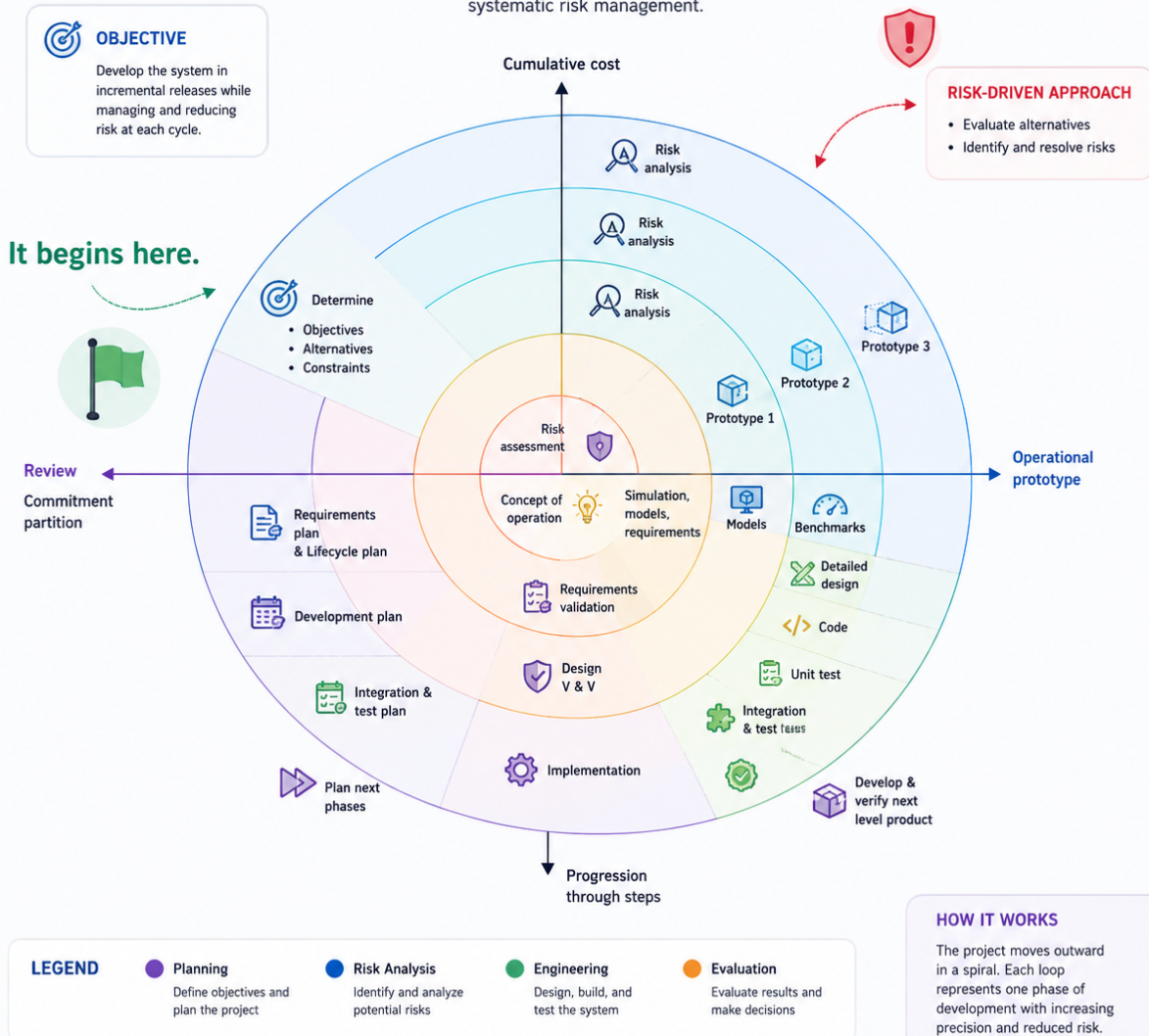
Explain how each cycle establishes objectives, reduces key risks, develops and validates the next level, then reviews and plans before greater commitment.

CORE LECTURE · RISK-DRIVEN LOOP

Each loop buys knowledge before buying commitment

Spiral Process Model

A risk-driven model that combines iterative development with systematic risk management.



Read the logic, not only the spiral: objectives and alternatives → risk assessment/reduction → development/validation → review and next planning.

Objectives

What are we trying to achieve and what alternatives exist?

Risk Reduction

What could fail, and what experiment or analysis reduces uncertainty?

Engineering

Develop and validate the next product level.

Evaluate & Plan

Review evidence and decide the next cycle.

No major commitment before reducing the **important risk**.

WORKED EXAMPLE

Medical AI decision support

The team wants an AI component but its accuracy on local patient data is uncertain. Before funding full integration, it isolates that technical risk, builds a limited prototype, evaluates error patterns with clinicians, compares alternatives, and only then decides whether to scale the investment.

MEMORY HOOK ·

Risk first, commitment second. Iteration alone does not make a process Spiral.

STUDY BOUNDARY Core lecture content ends above.

The section below is added clarification.

ADDED CLARIFICATION

Why Spiral matters

Best fit

Large, complex or high-consequence projects where technical, cost, safety or requirements uncertainty makes explicit risk reduction valuable.

Practical nuance

The model was highly influential in introducing risk-driven thinking, even though projects do not always use its published form literally.

TOPIC 04 · LESSON 08 · 8 OF 11 LESSONS

TOPIC 04 · CHAPTER 2 · LESSON 08

Agile Process Models

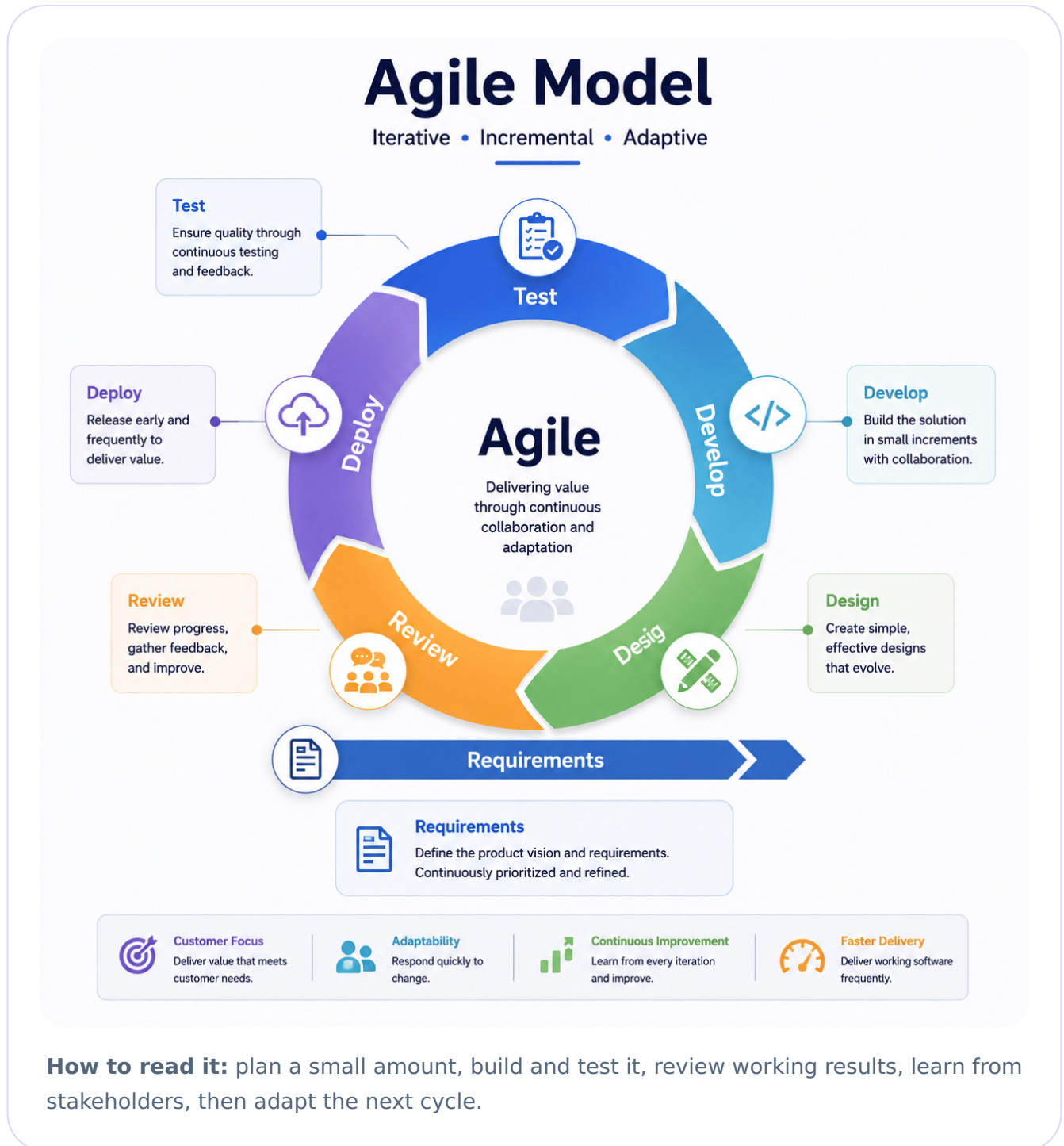
Agile turns short cycles into working increments, feedback and adaptation—not merely repeated activity.

LEARNING FOCUS

Explain how short iterations deliver useful software, expose feedback and adapt subsequent work to changing needs.

CORE LECTURE · VISUAL ANCHOR

Every cycle should create evidence and value



Plan

Build

Test

Review

Adapt

Iteration output: Working increment + stakeholder feedback + updated understanding → next iteration

✗ Iteration is not...

Doing the same unfinished work again and again.

✓ Iteration is...

A bounded cycle that produces a tested result, learning and a better-informed next decision.

WORKED EXAMPLE**Student services portal**

Deliver appointment booking first, test it, observe real use, review feedback with students and staff, then change reminders and waiting-list rules in the next short cycle.

REMEMBER

Agile is not “working fast.” It is disciplined adaptation through frequent working software, collaboration and feedback.

STUDY BOUNDARY Core lecture content ends above.

The section below is added clarification.

EXAM LENS**Recognize Agile logic****Look for**

Short iterations, working software, customer collaboration, repeated feedback and adaptation—even when requirements change late.

Do not infer

Agile does not mean no architecture, no documentation or no planning; it changes their timing and feedback cadence.

TOPIC 04 · LESSON 09 · 9 OF 11 LESSONS

TOPIC 04 · CHAPTER 2 · LESSON 09

Choosing the Right Model

Model selection is an engineering decision: choose the process logic that best manages the project's dominant uncertainty, constraints and risks.

LEARNING FOCUS

Evaluate project context and justify a suitable model using requirements stability, technology, complexity, reliability, schedule, cost, stakeholder visibility, skills, documentation and reuse.

DECISION ANCHORS

Start with the project—not your favorite model

**Requirements**

Stable and understood, or unclear and changing?

**Risk & Technology**

Familiar technology, or major technical/safety uncertainty?

**Feedback**

Do users need working software early to discover what they need?

**Control & Evidence**

How much documentation, traceability, visibility and management control are required?

CONTEXT MAP

Five model instincts

**Waterfall**

Stable requirements ·
planning · documentation

**Prototype**

Learn requirements early

**Evolutionary**

Grow the real product

**Spiral**

Reduce high risk before
commitment

**Agile**

Frequent change · short
feedback

Important: comparison tables are classroom heuristics, not scientific laws. Real projects often combine practices.

DECISION LAB

Choose — then justify

**Case A · Government payroll**

Requirements are mature, interfaces are controlled, formal approvals and documentation are heavy, and change is limited.

Strong fit: plan-driven / Waterfall characteristics because stability and evidence dominate.

**Case B · Experimental AI product**

User needs are unclear and the team must learn what features are valuable while exploring unfamiliar technology.

Strong fit: Prototyping + Evolutionary characteristics because learning and product growth dominate.

**Case C · Safety-critical innovation**

The project has high consequences, uncertain technology and expensive commitments.

Strong fit: Spiral / risk-driven logic because major uncertainty should be reduced before larger investment.

MEMORY HOOK

Choose the model that attacks the dominant uncertainty. Never answer with a model name only—justify it from project conditions.

STUDY BOUNDARY Core lecture content ends above.

The section below is added clarification.

EXAM LENS

Build an engineering argument

Good answer

“Spiral fits because unfamiliar technology creates high technical risk that should be reduced before major commitment.”

Weak answer

“Agile is best” or “Waterfall is old.” Popularity and age are not selection criteria.

TOPIC 04 · LESSON 10 · 10 OF 11 LESSONS

TOPIC 04 · CHAPTER 2 · LESSON 10

Agile Methods

Agile is a family of methods and practices, not one single recipe. Different methods emphasize different ways of planning, engineering, communicating and improving flow.

LEARNING FOCUS

Recognize major Agile methods and explain the distinctive emphasis each brings to an Agile project.

SIX METHODS · SIX HOOKS

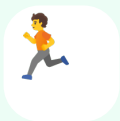
Same Agile family, different emphasis

**DSDM**

Emphasizes disciplined delivery within fixed time and resources, with active user involvement and prioritization.

**Feature Driven Development**

Organizes progress around small, client-valued features and short design/build cycles.

**Scrum**

Provides a concrete iterative management framework with short cycles, prioritized work and regular inspection/adaptation.

**Extreme Programming (XP)**

Strengthens technical discipline through frequent integration, testing, refactoring and close feedback.

**Crystal Clear****Lean Development**

Emphasizes people, communication and tailoring the process to team size and project criticality.

Focuses on reducing waste, improving flow, delivering value and avoiding unnecessary work or delay.

REAL TEAMS COMBINE PRACTICES

Agile methods are not mutually exclusive boxes



Scrum

Planning cadence & team workflow



XP

Testing, integration & refactoring discipline



Lean

Reduce waiting, handoffs & waste

THREE MEMORY ANCHORS

What should you remember?



Scrum = cadence

Think iterations, prioritized work, reviews and adaptation.



XP = engineering discipline

Think frequent testing, integration and refactoring.



Lean = flow

Think value, waste reduction and shorter delays.

WORKED EXAMPLE

Product team

A team might use Scrum for planning and coordination, XP practices for testing/refactoring, and Lean ideas to reduce waiting and handoffs. That combination is not a contradiction—it is a practical use of complementary Agile ideas.

REMEMBER

Agile principles can be implemented through different methods and combined practices. The method name matters less than understanding what problem each practice is trying to solve.

STUDY BOUNDARY Core lecture content ends above.

The section below is added clarification to make the concept easier to understand and revise.

EXAM LENS

Recognize the dominant emphasis

If the scenario emphasizes...

Iterations and team workflow → think Scrum.
Strong technical practices → think XP. Waste and flow → think Lean.

Do not confuse

“Agile” is the broad family. Scrum, XP, DSDM, FDD, Crystal and Lean are methods or approaches within that broader space.

TOPIC 04 · LESSON 11 · 11 OF 11 LESSONS

TOPIC 04 · CHAPTER 2 · LESSON 11

Software Paradigms

A paradigm is a way of seeing the software problem. Change the lens, and the same system is structured around different primary ideas.

LEARNING FOCUS

Distinguish procedural, object-oriented and data-oriented paradigms and connect paradigm, methodology and process without confusing them.

THREE LENSES

What becomes the center of attention?

**Procedural**

Functions, procedures, operations and control flow.

Question: What steps happen?

**Object-Oriented**

Objects that combine state, responsibility and behavior.

Question: Who owns behavior?

**Data-Oriented**

Data structures, information movement and transformations.

Question: What data exists and changes?

PARADIGM ≠ METHODOLOGY

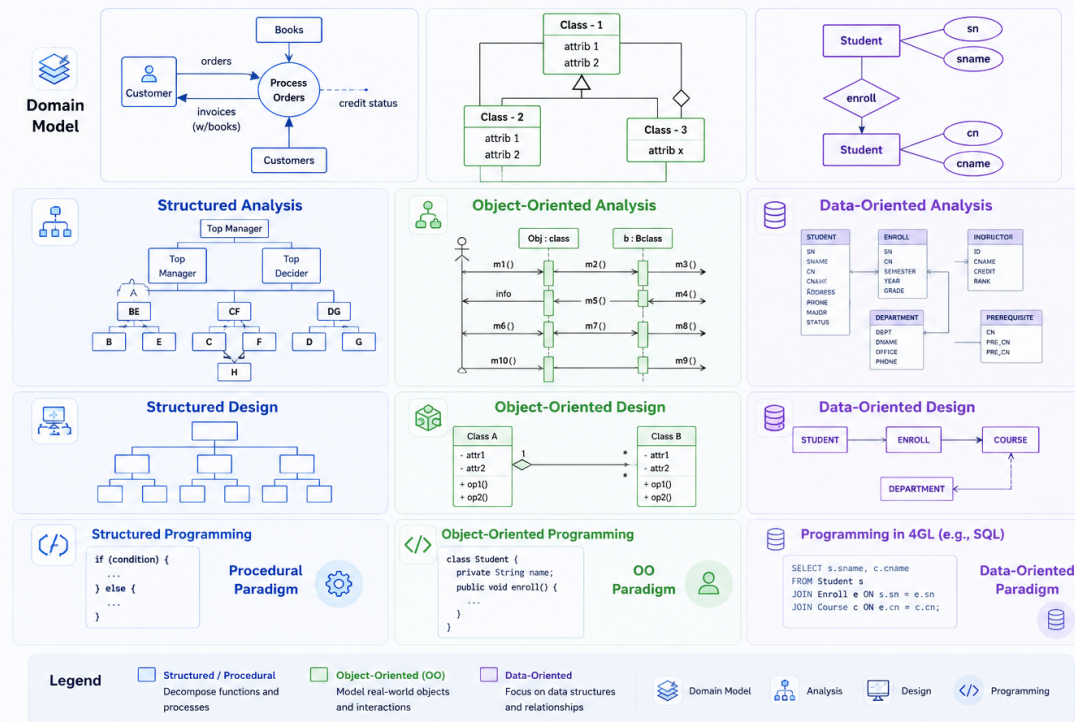
Lens versus route

Paradigm is the conceptual lens used to view and structure software reality. **Methodology** organizes how development work is carried out. They can influence one another, but they are not synonyms.

Paradigm and Methodology — visual reference

Paradigm and Methodology

Different ways to analyze, design and build software systems



TOPIC 04 · BIG PICTURE

See the whole chapter as one engineering argument

WHY? Development challenges create uncertainty, coordination, change and risk.



WHAT STRUCTURES THE WORK? Software process defines phases, artifacts, milestones and criteria.



HOW IS WORK PERFORMED? Methodology defines steps, techniques, representations and procedures.

**CONCEPT CHECK****Can you explain these without looking back?****01**

What is the difference between a process and a methodology?

02

Why is throwaway prototyping different from evolutionary development?

03

Why is Spiral called risk-driven rather than simply iterative?

04

When can Waterfall be a reasonable engineering choice?

05

Why is there no universally “best” process model?

Mastery test: if you can answer all five using project conditions and engineering reasoning—not memorized slogans—you understand the chapter’s core logic.

FINAL MEMORY HOOK

Challenge → Process → Methodology → Model → Selection → Paradigm. That is the mental map of Topic 04.